Smart Village Scenario

Submitted By

Student Name	Student ID
Shariar Ahamed Ripon	0242310005101019
Md. Moniruzzaman Rifat	0242310005101020
Sultana Asma Islam	0242310005101682

LAB PROJECT REPORT

This Report Presented in Partial Fulfillment of the course

CSE412/422: Computer Graphics Lab in the Department of Computer Science and Engineering

DAFFODIL INTERNATIONAL UNIVERSITY

Dhaka, Bangladesh

23/04/2026

DECLARATION

We hereby declare that this lab project has been done by us under the supervision of Khandokar Nosiba Arifin, Lecturer, Department of Computer Science and Engineering, Daffodil International University. We also declare that neither this project nor any part of this project has been submitted elsewhere as lab projects.

Submitted To:

Nosiba 23/04/26

Khandokar Nosiba Arifin
Lecturer
Department of Computer Science and Engineering
Daffodil International University

Submitted By

Shariar

Shariar Ahamed Ripon
ID: 0242310005101019
Dept. of CSE, DIU

Rifat

Md Moniruzzaman Rifat
ID: 0242310005101020
Dept. of CSE, DIU

Sultana

Sultana Asma Islam
ID: 0242310005101682
Dept. of CSE, DIU

COURSE & PROGRAM OUTCOME

The following course have course outcomes as following:

Table 1: Course Outcome Statements

CO1	Design and implement fundamental computer graphics primitives and algorithms to construct 2D graphical scenes for real-life visualization problems having complex engineering attribute
CO2	Apply appropriate OpenGL programming techniques, geometric transformations, and graphics algorithms to solve complex real-world visualization problems using modern engineering tools.
CO3	Perform effectively as an individual and as a member of a diverse team to design, document, and implement computer graphics–based projects.
CO4	Create and present a complete computer graphics project by explaining complex engineering activities through effective reports, demonstrations, and presentations.

Mapping Course Outcome (COs) with the Teaching-Learning and Assessment Strategy

The mapping justification of this table is provided in section **4.2.1**, **4.2.2** and **4.2.3**.

Table 2: Mapping of CO, PO, Blooms, KP and CEP

CO	**POs**	**Teaching Learning Activities**	**Assessment Strategy**	**Learning Domains**	**Knowledge Profile**	**Complex Engineering Problem**	**Complex Engineering Activities**
CO1	PO1	TLA1 (Lab exercises)	Lab performance, Lab report	C2	K1, K3	EP1,EP4	
CO2	PO5	TLA2 (Algorithm implementation)	Lab performance, Lab report	C3, C4	K6	EP1, EP4, EP3	
CO3	PO9	TLA3 (Group project)	Lab project	P3, A2			
CO4	PO10	TLA4 (Presentation & Viva)	Project & Viva	C6, P3, A2			EA1, EA3

Table of Contents

Chapter 1

Introduction

This chapter provides an overview of the Computer Graphics project, including its objectives, scope, and significance. It establishes the foundation by explaining the project's theme, visual concept, and the goals it aims to achieve through graphical modeling and animation.

1.1 Introduction

Computer Graphics is an important field of computer science that focuses on creating visual content using programming and mathematical models. In modern applications, graphics are widely used in games, simulations, education, and design systems. This project presents a 2D animated village scenery using OpenGL, where different real-world elements are visually represented through code. [1] The main idea of this project is to simulate a rural village environment that includes various natural and human-made objects such as sky, sun, moon, trees, houses, river, and people. In addition to static objects, several animated elements are included to make the environment more realistic. These include moving clouds, rotating windmill, flowing river, walking humans, and dynamic weather conditions like rain. Another important feature of this project is the day and night transition system, where the sun moves across the sky and is replaced by the moon, creating a natural cycle. During night mode, stars appear and lights turn on, making the scene more realistic. This project uses OpenGL functions to draw shapes and apply transformations such as translation, rotation, and scaling. Overall, the project demonstrates how simple graphical primitives can be combined to build a complex and dynamic environment.

1.2 Motivation

The motivation behind developing this project comes from the need to understand how theoretical concepts of computer graphics can be applied in real-world scenarios. Many students learn algorithms like DDA and Midpoint Circle in theory but do not get the opportunity to implement them in practical projects. This project provides a platform to apply those concepts effectively. Another motivation is to create a visually appealing and interactive system that represents a real-life environment. A village scenery is chosen because it contains a variety of natural and man-made elements, making it suitable for demonstrating different graphics techniques. By building such a project, one can learn how complex scenes are created from simple shapes. The project also aims to improve programming skills in C/C++ and understanding of OpenGL. It encourages problem-solving, logical thinking, and creativity. Handling multiple animations at once, managing user inputs, and synchronizing different components require careful planning and execution. Furthermore, this project can serve as a foundation for future work in game development, simulation systems, and graphical user interfaces. It helps students gain confidence in developing interactive applications and understanding how graphics systems work internally.

1.3 Objectives

The primary objective of this project is to design and implement a complete 2D animated village environment using OpenGL. The project aims to combine different elements of computer graphics into a single interactive system. One of the key objectives is to simulate a real-world rural environment with both static and dynamic

objects. Another important objective is to implement a day and night cycle, where the sun and moon move across the sky and trigger changes in lighting and environment. This adds realism and demonstrates how conditions can be dynamically controlled in a graphics system. The project also focuses on adding various types of animations. These include moving clouds, rotating windmill, walking humans, flowing river, and rainfall. Each animation is controlled using variables and updated continuously to create smooth motion. Additionally, the project aims to apply different 2D transformations, such as translation for movement, rotation for spinning objects, and scaling for resizing objects like the boat. These transformations are essential concepts in computer graphics. Another objective is to include user interaction, allowing the user to control certain aspects of the scene using keyboard input, such as turning rain on or off and scaling the boat.

1.4 Feasibility Study

The feasibility of this project can be analyzed from technical, economic, and operational perspectives. From a technical point of view, the project is highly feasible because it uses OpenGL, which is a well-established and widely supported graphics library. The required algorithms and transformations are simple and can be implemented using basic programming knowledge. From an economic perspective, the project does not require any additional cost. It uses free tools such as OpenGL and GLUT, which are available for all major platforms. The hardware requirements are minimal, and the project can run smoothly on standard computers without the need for high-performance graphics cards. Operationally, the project is easy to develop and maintain. The code is modular, with separate functions for different objects and animations. This makes it easier to update or extend the project in the future. The user interface is simple, and interaction is handled through keyboard inputs, which are easy to implement and use. Time feasibility is also favorable, as the project can be completed within a reasonable timeframe with proper planning. The simplicity of the algorithms and the availability of resources make the development process efficient.

1.5 Gap Analysis

In many basic computer graphics projects, the focus is mainly on drawing static objects such as lines, circles, and polygons. These projects often lack interactivity and dynamic behavior, which are essential for creating realistic simulations. As a result, students may not fully understand how graphics systems work in real-world applications. This project addresses these limitations by introducing multiple advanced features. Unlike traditional projects, this system includes real-time animation, where objects continuously move and change over time. For example, clouds move across the sky, the boat travels along the river, and humans walk on the path. Another gap in existing projects is the absence of environmental changes. This project includes a day-night transition system, which dynamically changes the appearance of the scene. It also includes weather effects such as rain, which can be controlled by the user. User interaction is another area where many projects fall short. This project allows users to interact with the scene using keyboard inputs, making it more engaging and functional. By combining multiple features such as animation, transformation, interaction, and environmental changes, this project provides a more complete and practical understanding of computer graphics compared to traditional static projects.

1.6 Project Outcome

The outcome of this project is a fully functional and interactive 2D animated village scenery developed using OpenGL. The system successfully demonstrates the integration of various computer graphics concepts, including algorithms, transformations, and animation techniques. One of the key outcomes is the successful implementation of a dynamic environment, where elements such as the sun, moon, clouds, and rain change over time. The day and night cycle works smoothly, providing a realistic representation of natural conditions.

The addition of stars and lighting effects during night mode enhances the visual experience. Another important outcome is the inclusion of multiple animations. Objects like the windmill and gear rotate continuously, while clouds and humans move across the scene. The boat moves along the river and can be resized using keyboard input, demonstrating the use of scaling transformation. The project also provides an interactive experience, allowing users to control certain features such as rain and object size. This improves user engagement and demonstrates the practical application of input handling in graphics systems.

Chapter 2

System Architecture/ Scene Description & Architecture

This chapter describes the overall scene design and architecture of the Computer Graphics project. It outlines the main graphical components, their interactions, and the technologies used to create an efficient, visually appealing, and animated OpenGL-based scene.

2.1 Requirement Analysis & Design Specification

From the requirement analysis, the project needs basic software tools such as OpenGL, GLUT, and a C compiler to create and run the graphical application. It also requires a standard computer system with minimal hardware specifications. In addition, knowledge of computer graphics concepts like algorithms, transformations, and animation is necessary. From the design perspective, the project is structured into different parts such as sky, land, river, houses, trees, and moving objects like clouds, humans, and boat. Each component is designed separately and then combined to form a complete scene. The design also includes animation logic and user interaction, ensuring smooth performance and a realistic environment.

2.1.1 Scene Overview

Describe the scene you have built (e.g., village, traffic, interior design).

2.1.2 Object Components

The project is designed as a layered 2D scene where different elements are arranged based on their position and importance. The topmost layer represents the sky, which changes color depending on whether it is day or night. The sky contains objects like the sun, moon, clouds, and stars, which are animated to create a dynamic effect. Below the sky, the middle section includes houses, trees, fences, and a windmill, which represent the village environment. These objects are mostly static but contribute to the overall realism of the scene. The windmill and gear are animated to show rotational motion. The lower section contains the river, boat, and walking path. The river changes color based on the time of day, and the boat moves continuously across it. The walking path is used for human movement, where multiple characters walk at different speeds. Additional features such as rain, solar lights, and human movement are integrated into the scene. The entire design is structured in such a way that each component is independent but works together to create a cohesive environment.

2.2 Use of Modern Tools

This project uses modern and widely accepted tools in computer graphics development. The primary tool is OpenGL, which is a cross-platform graphics API used for rendering 2D and 3D graphics. It provides functions to draw shapes, apply transformations, and manage rendering efficiently. Another important tool used is GLUT (Graphics Utility Toolkit). GLUT simplifies the process of creating windows, handling keyboard input, and managing the rendering loop. It acts as a bridge between the system and OpenGL, making development easier. The programming language used is C/C++, which provides control over memory and performance. It is commonly used in graphics programming due to its efficiency and compatibility with OpenGL. These tools together provide a strong foundation for developing interactive graphics applications. They are widely used in academic and industrial environments, making this project relevant and practical

2.2 Algorithm Used

The project uses two fundamental algorithms from computer graphics: the DDA (Digital Differential Analyzer) algorithm and the Midpoint Circle algorithm. These algorithms are essential for drawing basic shapes efficiently. The DDA algorithm is used to draw straight lines by calculating intermediate points between two given endpoints. It works by incrementing the x and y coordinates step-by-step based on the slope of the line. In this project, DDA is used to draw elements like the walking path and boundaries. The Midpoint Circle algorithm is used to draw circles by calculating points using symmetry. Instead of computing every point, it determines the next point based on a decision parameter, which reduces computational complexity. This algorithm is used to draw objects such as the sun, human heads, and tree leaves. By implementing these algorithms, the project demonstrates how mathematical concepts can be applied to generate graphical shapes efficiently and accurately.

2.4 2D Transformations Applied

Transformations are an essential part of computer graphics, and this project uses three main types: translation, rotation, and scaling. Translation is used to move objects from one position to another. In this project, translation is applied to moving elements such as clouds, humans, and the boat. By continuously updating position variables, smooth motion is achieved. Rotation is used to rotate objects around a fixed point. The windmill and gear use rotation to simulate real-world movement. The rotation angle is updated continuously to create a spinning effect. Scaling is used to change the size of objects. In this project, scaling is applied to the boat, allowing the user to increase or decrease its size using keyboard input. This demonstrates how objects can be resized dynamically. These transformations are implemented using OpenGL functions such as glTranslatef, glRotatef, and glScalef. Together, they help create a dynamic and interactive scene.

2.5 Animation Logic

The animation in this project is achieved through continuous updates and rendering. The main idea behind the animation logic is to change object properties over time and redraw the scene repeatedly. The function glutPostRedisplay() is used to refresh the display continuously. In each frame, the positions of moving objects such as clouds, humans, and the boat are updated. This creates the illusion of motion. Conditional logic is used to control environmental changes. For example, when the sun moves beyond a certain point, the system switches to night mode, and the moon appears. Similarly, rain can be turned on or off using keyboard input. Arrays are used to simulate rain, where each raindrop has its own position and falls downward over time. Star blinking is achieved using a counter variable that toggles visibility.

Chapter 3

Implementation and Results

This chapter describes how the project is implemented in code and presents the final output and performance of the system.

3.1 Implementation

The implementation of this project is done using a modular approach. Each object in the scene, such as trees, houses, clouds, and humans, is created using separate functions. This improves readability and makes the code easier to manage. Global variables are used to control animation properties such as position, speed, and rotation angle. These variables are updated continuously to create motion. Keyboard input is handled using a dedicated function, allowing users to interact with the scene. The OpenGL functions glBegin, glEnd, and various primitives like GL_POLYGON, GL_LINES, and GL_POINTS are used to draw shapes. Transformations are applied using built-in functions to control movement and rotation.The implementation focuses on simplicity and clarity while maintaining functionality and performance.

3.2 Output

The output of the project is a visually rich and dynamic village scene. The display shows a complete environment with sky, land, river, houses, trees, and moving objects. During the day, the sky appears bright with the sun, and during the night, it becomes dark with the moon and stars. The transition between day and night is smooth and visually appealing. Animations such as moving clouds, walking humans, rotating windmill, and flowing rain enhance the realism of the scene. The boat moves across the river and can be resized by the user.

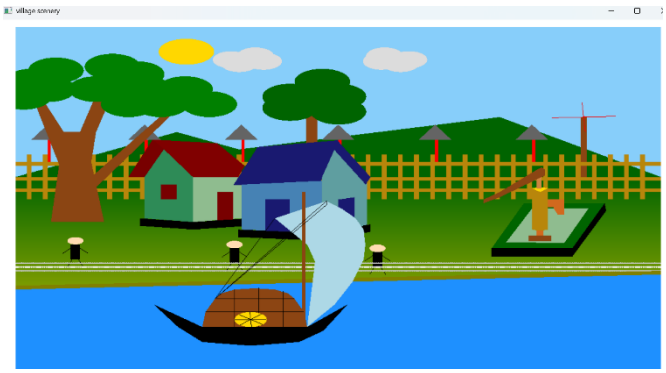

Fig 3.2.1: Day Scenario

Fig 3.2.2: Night Scenario

Fig 3.2.3: Rain Scenario

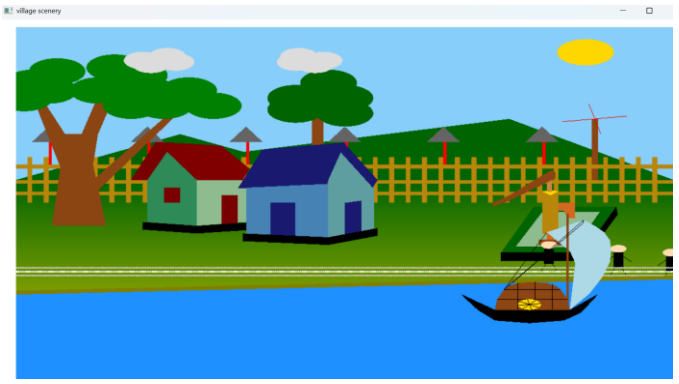

Fig 3.2.4: Boat Scaling

3.3 Discussion

The project effectively demonstrates the integration of multiple computer graphics concepts. The combination of algorithms, transformations, and animation creates a realistic and engaging scene. One of the strengths of the project is its modular design, which makes it easy to understand and extend. The use of simple algorithms ensures efficiency and performance. However, there are some challenges, such as managing multiple animations simultaneously and ensuring smooth transitions between different states. These challenges were addressed through careful planning and testing. The project provides a solid foundation for further

improvements and can be extended with additional features.

Chapter 4

Engineering Standards and Mapping

This chapter discusses the engineering standards followed throughout the project and maps the project activities to the course and program outcomes. It highlights how the project meets academic and professional requirements.

4.1 Impact on Society, Environment and Sustainability

4.1.1 Impact on Life

This project has a positive impact on learning and understanding computer graphics concepts in a practical way. It helps students visualize how real-world environments can be created using programming and mathematical logic. Instead of only studying theory, users can see how objects move, interact, and change dynamically. This improves both conceptual clarity and interest in the subject. The project also enhances problem-solving skills, as it involves managing multiple animations and interactions simultaneously. Students can learn how to break down complex systems into smaller components and implement them step by step. This approach is useful not only in graphics but also in other areas of software development. Additionally, such simulation-based projects can be used in educational tools, where visual representation makes learning easier and more engaging. For example, similar techniques can be applied in training systems, virtual environments, and interactive applications.

4.1.2 Impact on Society & Environment

The project represents a rural village environment in a digital form, which can be useful for educational and awareness purposes. It allows users to understand the structure and components of a village, including natural elements like trees and rivers, as well as human-made structures like houses and windmills. From a societal perspective, such simulations can be used in teaching and presentations to demonstrate how environments work. It can also inspire further development in areas like urban planning, environmental simulation, and virtual reality applications. Environmentally, the project does not have any negative impact, as it is purely digital and does not consume physical resources beyond basic computing power. It promotes awareness of natural elements such as weather, day-night cycles, and ecological balance. In the future, similar projects can be extended to simulate environmental changes, climate conditions, or disaster scenarios, which can be helpful for research and education.

4.1.3 Ethical Aspects

The project follows ethical guidelines as it is developed purely for educational and learning purposes. It does not include any harmful, offensive, or misleading content. All elements in the project are neutral and represent common aspects of a village environment. The use of open-source tools like OpenGL and GLUT also ensures that the project respects licensing and usage policies. No copyrighted or restricted materials are used without permission. Another ethical aspect is transparency. The project clearly demonstrates how algorithms and transformations are used, allowing others to learn and understand the process. It encourages knowledge sharing and collaboration among students. Additionally, the project does not misuse user data or involve any privacy concerns, as it does not collect or process personal information. It is a standalone application focused on visualization. Overall, the project aligns with ethical standards in software development and promotes

responsible use of technology.

4.1.4 Sustainability Plan

The sustainability of this project lies in its ability to be extended and reused in future developments. The modular design allows new features to be added without modifying the entire system. For example, additional objects, animations, or interactions can be integrated easily. The project can also be adapted for different purposes, such as educational tools, demonstration systems, or simulation environments. This makes it a reusable resource for future academic work. From a technical perspective, the use of lightweight tools ensures that the project remains efficient and does not require high computational resources. This makes it sustainable in terms of performance and accessibility. In the future, the project can be upgraded to include 3D graphics, advanced lighting, and more realistic physics. These improvements can enhance its usability and relevance.

4.2 Complex Engineering Problem

4.2.1 Mapping of Program Outcome

In this section, provide a mapping of the problem and provided solution with targeted Program Outcomes (PO's).

Table 4.1: Justification of Program Outcomes

POs	Justification of Mapping (Project Perspective)
PO1	This project applies mathematical and computational knowledge to create and transform graphical objects using OpenGL. It integrates geometry, coordinate systems, and matrix operations to solve complex visualization and animation problems in computer graphics. (PO1).
PO5	The project incorporates modern computer graphics tools and technologies such as OpenGL, FreeGLUT, and C++ programming to ensure the application of contemporary engineering practices and visualization techniques in real-world scenarios.
PO9	Through collaborative project work, every member was engaged in teamwork, task allocation, and documentation. (PO9).
PO10	Presenting the developed **computer graphics project**, preparing technical reports, and communicating ideas clearly (PO10)

4.2.2 Complex Problem Solving

The following table presents the mapping of the project with the **Complex Engineering Problem (CEP) attributes defined by the Washington Accord framework**. The attributes that are addressed in this project are marked (✓).

Table 4.2: Mapping with complex problem solving.

EP1 Dept of Knowledge	**EP2** Range of Conflicting Requirements	**EP3** Depth of Analysis	**EP4** Familiarity of Issues	**EP5** Extent of Applicable Codes	**EP6** Extent Of Stakeholder Involvement	**EP7** Inter-dependence
✓		✓	✓			

EP1: This project demonstrates a good understanding of basic Computer Graphics concepts. It applies algorithms like DDA and Midpoint Circle for drawing shapes, and uses 2D transformations such as translation, rotation, and scaling. The project also includes animation and real-time updates, showing knowledge of how dynamic systems work. Features like day-night transition, rain, and user interaction highlight the ability to combine multiple concepts.

EP3: I This project shows analytical thinking by breaking a complex village scene into smaller components such as sky, land, river, and moving objects. Each part is analyzed and implemented separately to ensure correct behavior. Different animations are carefully managed using variables and conditions, such as day-night switching and movement control. Performance and rendering flow are also considered to maintain smooth output.

EP4: This project demonstrates familiarity with common issues in computer graphics programming. While developing the system, challenges such as managing multiple animations at the same time, ensuring smooth rendering, and handling screen refresh were considered and addressed. Another common issue is object synchronization, where moving elements like clouds, boats, and humans must move independently without affecting each other. The project handles this using separate variables and update logic. Memory and performance concerns are also considered, especially when using arrays for rain effects and continuous animation loops. Proper structuring helps avoid unnecessary overhead

4.2.3 Complex Engineering Activities

Table 4.3 presents the mapping of the project with the **Complex Engineering Activities (CEA) attributes**. The attributes relevant to this project are marked (✓).

Table 4.3: Mapping with complex engineering activities.

EA1	EA2	EA3	EA4	EA5
✓		✓		

EA1: This project uses a limited but effective set of resources for development. It mainly relies on OpenGL and GLUT libraries for graphics rendering and window management, along with a C/C++ compiler for coding and execution.

Standard computer hardware is sufficient to run the project without requiring high-end GPU or advanced systems. No external datasets or complex software tools are needed

EA3: This project shows creativity by combining multiple graphical elements into a single animated village scene. It includes day-night transition, moving objects, rain effects, and user interaction in one system.

The integration of different animations and environmental changes makes the scene more realistic and engaging compared to basic static graphics projects..

Chapter 5

Conclusion

5.1 Summary

This project successfully demonstrates the creation of a 2D animated village scenery using OpenGL. It integrates various concepts of computer graphics, including algorithms, transformations, and animation techniques. The system includes multiple features such as day-night transition, moving objects, weather effects, and user interaction. These features work together to create a realistic and dynamic environment. The project also highlights the importance of planning, design, and implementation in developing

5.2 Limitation

Despite its success, the project has some limitations. It is limited to 2D graphics and does not include 3D visualization, which could enhance realism. The graphical quality is basic, as it uses simple shapes and colors. [1] Another limitation is the absence of sound effects, which could improve the user experience. The interaction is also limited to keyboard input and does not include advanced controls. [2] Additionally, the project does not use advanced lighting or shading techniques, which could make the scene more realistic.

5.3 Future Work

In the future, the project can be improved in several ways. One major enhancement is the addition of 3D graphics, which would provide depth and realism. Advanced lighting and shading techniques can also be applied. Sound effects can be added to simulate real-world conditions, such as rain sounds or windmill noise. More interactive features can be included, such as mouse control or menu systems. Additional objects and animations can be added to make the scene more complex and engaging. The project can also be optimized for better performance.

References

[1] D. G. Aliaga, "3D design and modeling of smart cities from a computer graphics perspective," *International Scholarly Research Notices,* vol. 2012, no. Wiley Online Library, p. 728913, 2012.

[2] K. Pulli, Mobile 3D graphics: with OpenGL ES and M3G, 2007.

[3] P. Suanpang, "Extensible metaverse implication for a smart tourism city," *Sustainability,* vol. 14, no. MDPI, p. 14027, 2022.

Team Work Contribution:

Name	Student Id	Contribution
Shariar Ahamed Ripon	0242310005101019	Field, Cloud, Fence, Tube well, Sun & moon movement condition, Windmill ,Apple (circle algorithm) Rain
Md. Moniruzzaman Rifat	0242310005101020	River, Sky, Boat(Movement), Boat (scaling), Gear (rotation), Day / mood full screen & star blink, Final output, Car (2d translation)
Sultana Asma Islam	0242310005101682	Sun, Tree, House, Solar system, Walking path (DDA algorithm), People add